



Pizza Express

Sistema de Gestión de Pedidos e Inventario

Estrategia de gestión del proyecto

Proyecto de Ingeniería de Software
Católica del Norte Fundación Universitaria



- **Proyecto:** Pizza Express — Sistema de Gestión de Pedidos e Inventario
- **Institución:** Católica del Norte Fundación Universitaria
- **Asignatura:** Ingeniería de Software
- **Duración:** 1 mes (4 sprints de una y dos semanas)

1. Definición del proyecto

Objetivo general

Digitalizar la toma de pedidos y el control de inventario en tiempo real, para **reducir errores en los pedidos** y **evitar quiebres de ingredientes durante el servicio**, entregando un MVP (Producto Mínimo Viable) funcional en un mes.

El foco del trabajo es la **planeación y gestión del proyecto**; el prototipo funcional es la demostración de que lo planeado se puede construir.

Alcance de la versión 1

En alcance	Fuera de alcance
Pedidos en mesa, para llevar y a domicilio	Pagos en línea
Inventario con descuento automático por pizza	Aplicación de repartidores con GPS
Alertas de stock mínimo	Programa de fidelización
Reportes básicos de ventas	

Dejar el alcance cerrado y escrito desde el primer sprint fue la principal medida contra el *scope creep*, uno de los riesgos identificados.

Justificación

En una pizzería, el error más caro no es equivocarse en un pedido, sino **vender lo que no se puede preparar**: cuando un insumo se agota en pleno servicio, el pedido ya está cobrado y el cliente esperando. Por eso el sistema no se limita a registrar ventas: valida el inventario antes de aceptar cada pedido.

2. Stakeholders

Stakeholder	Interés en el proyecto	Cómo lo atiende el sistema
Cliente / comensal	Recibir el pedido correcto y a tiempo	Pedido registrado con modalidad, notas y seguimiento de estado
Cajero / mesero	Tomar pedidos rápido y sin errores	Formulario con el menú, precios visibles y aviso inmediato si algo no se puede preparar
Pizzero / cocina	Saber qué preparar y en qué orden	Tablero de cocina por estados, con las notas del cliente
Administrador / dueño	Controlar inventario, precios y ventas	Gestión de inventario, precios del menú, alertas y reportes
Proveedores	Reabastecer a tiempo (actor externo)	Alertas de stock mínimo que anticipan el pedido de compra
Equipo de desarrollo	Construir y entregar el sistema	Repositorio, tablero de tareas y documentación

3. Metodología

Ágil (Scrum) frente a Cascada

Criterio	Ágil — Scrum  elegida	Cascada  descartada
Entregas	Incrementales, por sprints	Una sola entrega al final
Requisitos	Se adapta a requisitos que cambian	Congelados desde el inicio
Retroalimentación	Temprana y continua	Solo al final
Corrección de errores	En cualquier sprint	No permite corregir hasta el final
Riesgo	Bajo: cada sprint entrega algo usable	Alto si algo cambia
Ajuste al proyecto	Ideal para equipo pequeño y plazo corto	Demasiado rígida para un mes de trabajo

Decisión: Scrum. Con un mes de plazo y requisitos que se refinaban sobre la marcha, entregar un MVP y ajustarlo cada semana era la única forma realista de llegar. Con Cascada, un cambio en la definición del alcance en la semana 3 habría obligado a rehacer el análisis completo.

Ceremonias

Ceremonia	Frecuencia	Propósito
Planeación del sprint	Al inicio de cada sprint	Elegir del backlog qué entra en el sprint
Reunión diaria	Diaria, breve	Qué hice, qué haré, qué me bloquea
Revisión del sprint	Al cierre de cada sprint	Mostrar el incremento terminado
Retrospectiva	Al cierre de cada sprint	Qué mantener, qué mejorar

4. Equipo y roles

Integrantes

- Nini Yohana Grandas Tellez
- Emanuel García Gutiérrez
- Juan David Cumbe Hernández
- Johan Esteban Rodríguez Valencia
- Oscar Mario Montoya Caro

Roles Scrum

Rol	Responsabilidad
Product Owner	Prioriza el backlog y representa al dueño de la pizzería ante el equipo
Scrum Master	Facilita las ceremonias, organiza los sprints y elimina bloqueos
Equipo de desarrollo	Frontend, backend y base de datos: construyen el MVP
QA / Pruebas	Valida cada incremento antes de darlo por terminado. Es un rol rotativo : cada sprint lo asume un integrante distinto

Se decidió rotar el rol de QA para que nadie probara su propio trabajo y para que todo el equipo conociera el sistema completo, no solo su parte.

5. Cronograma

Cuatro sprints distribuidos en el mes:

Sprint	Semana	Nombre	Entregable
1	Semana 1	Definición	Objetivo, alcance, stakeholders y riesgos. Tablero de Trello y repositorio montados
2	Semana 1	Metodología y cronograma	Justificación de Ágil, roles y diagrama de Gantt en Excel. Backlog poblado en Trello
3	Semana 2	Stack y modelado	Justificación de las herramientas y diagramas UML: casos de uso, clases y entidad-relación
4	Semana 2	Presentación y cierre	Integración de todo y entrega del proyecto

Priorización del backlog (MoSCoW)

Prioridad	Historias
Must have	Registrar pedidos en las tres modalidades · Descontar el inventario por receta · Bloquear la venta si falta un insumo · Alertas de stock mínimo
Should have	Reportes de ventas · Tablero de cocina · Gestión de precios del menú
Could have	Control de acceso por rol · Filtros y búsqueda en las listas
Won't have (v1)	Pagos en línea · GPS de repartidores · Fidelización

Clasificar el backlog con MoSCoW permitió decidir sin discusión qué se dejaba fuera cuando el tiempo apretó.

6. Gestión de riesgos

Riesgo	Prob.	Impacto	Mitigación planeada	Cómo quedó implementada
Quiebre de un ingrediente en pleno servicio	Alta	Alto	Inventario en tiempo real, alerta de stock mínimo y bloqueo de venta si falta un insumo clave	<code>verificar_disponibilidad</code> impide crear el pedido y explica qué falta; <code>obtener_alertas</code> avisa antes de llegar a cero. Verificado en los casos CP-07 y CP-14 del plan de pruebas
El alcance se infla (scope creep)	Alta	Alto	Alcance v1 cerrado y backlog priorizado con MoSCoW	La sección «fuera de alcance» se respetó: no se implementaron pagos, GPS ni fidelización
Plazo ajustado	Alta	Alto	Metodología ágil con un MVP entregable por sprints	Cuatro sprints con entregable propio; el sistema fue usable desde el primer incremento
Curva de aprendizaje del equipo	Media	Medio	Usar herramientas ya conocidas por el equipo	Python, SQLite, Trello, GitHub y draw.io; ninguna herramienta nueva que aprender
Pérdida de pedidos o datos	Baja	Alto	Control de versiones con Git y respaldos periódicos	Repositorio en GitHub; los pedidos se escriben en una transacción con <code>rollback</code> , de modo que nunca queda un pedido a medias

7. Herramientas y su justificación

Herramienta	Para qué	Por qué esa y no otra
Trello	Gestión de tareas	Kanban visual, gratuito y de curva mínima; el enlace se comparte fácil. Frente a Jira o Asana, es más simple para un equipo pequeño
GitHub + Git	Repositorio de código	Control de versiones y colaboración por ramas. Gratuito y además queda como portafolio
draw.io	Modelado UML	Diagramas de casos de uso, clases y entidad-relación. Gratuito y exporta imágenes directo a la presentación
Python	Desarrollo del prototipo	Stack conocido por el equipo y rápido de desarrollar, fácil de justificar para el módulo de inventario
Excel	Cronograma	Diagrama de Gantt con los 4 sprints y sus historias

Nota sobre el stack de desarrollo

En la presentación se propuso **Python + Django**. Al construir el prototipo se mantuvo Python pero se usó **Flask**, un framework del mismo ecosistema y más pequeño, porque para el tamaño de este MVP deja el código a la vista, necesita una sola dependencia y permite mostrar el modelo de datos en vez de esconderlo tras un ORM. La justificación completa está en la documentación técnica.

8. Tablero de tareas

El flujo de trabajo en Trello tiene cinco columnas:

Backlog → Por hacer → En progreso → En pruebas → Hecho

La columna **En pruebas** es la que hace efectivo el rol de QA: ninguna tarea pasa a *Hecho* sin que otro integrante la haya validado.

Enlace del tablero:

<https://trello.com/invite/b/6a80053921a065f7175fe156/ATTIf22e678827ee0cf137ae723dc34edeed77217503/pizza-express>

Definición de «Hecho»

Una tarea solo se mueve a *Hecho* cuando cumple todo lo siguiente:

1. La funcionalidad está implementada y probada por su autor.
 2. El código está subido al repositorio.
 3. Otro integrante (rol QA) la validó en la columna *En pruebas*.
 4. Si aplica, tiene una prueba automática que la cubre.
 5. La documentación afectada quedó actualizada.
-

9. Gestión de la configuración

- **Repositorio:** GitHub, con el historial completo del proyecto.
 - **Ramas:** trabajo por rama y unión a la rama principal una vez validado.
 - **Commits:** mensajes descriptivos en español, indicando qué cambia y por qué.
 - **Datos:** la base de datos (`datos/*.db`), el entorno virtual (`.venv/`) y los archivos temporales de Python están excluidos con `.gitignore`, para que el repositorio solo contenga código y documentación.
 - **Reproducibilidad:** cualquier integrante puede clonar el repositorio y tener el sistema funcionando con datos de ejemplo en tres comandos, sin pedirle la base de datos a nadie.
-

10. Gestión de la calidad

La calidad se controló en tres frentes:

Frente	Mecanismo
Código	Estructura en tres capas, nombres y comentarios en español, y una única responsabilidad por módulo
Funcionalidad	18 pruebas automáticas sobre la lógica de negocio y 17 casos de prueba de sistema, con matriz de trazabilidad contra los requerimientos
Proceso	Columna <i>En pruebas</i> en Trello y definición de «Hecho» acordada por el equipo

El detalle está en el plan de pruebas.

11. Entregables

Entregable	Descripción	Estado
Presentación dinámica	Todo el proceso: definición, metodología y stack	Entregado
Cronograma en Excel	Diagrama de Gantt con los 4 sprints y sus historias	Entregado
Tablero de Trello	Enlace con el backlog y las tareas por sprint	Entregado
Prototipo funcional	Aplicación web Pizza Express	Entregado
Plan de pruebas	plan-de-pruebas.md	Entregado
Documentación técnica	documentacion-tecnica.md	Entregado
Manual de usuario	manual-de-usuario.md	Entregado
Estrategia de gestión	Este documento	Entregado

La entrega se realiza por el espacio habilitado para la actividad, y el código y la documentación quedan en el repositorio del proyecto.

12. Lecciones aprendidas

1. **Cerrar el alcance por escrito desde el sprint 1 fue lo que salvó el plazo.** Cada vez que surgió una idea nueva (pagos en línea, domiciliarios con GPS), la lista de «fuera de alcance» permitió decir que no sin discusión y anotarla para una versión 2.
2. **El riesgo mejor mitigado fue el del quiebre de inventario**, porque se tradujo en una regla concreta del sistema y no en una buena intención: el pedido no se guarda si falta un insumo.
3. **Rotar el rol de QA** hizo que aparecieran defectos que el autor de cada parte no veía.
4. **Escoger herramientas ya conocidas** eliminó la curva de aprendizaje y dejó todo el tiempo disponible para el producto.
5. Para una próxima versión, convendría **automatizar también las pruebas de interfaz**: los 17 casos de sistema se tuvieron que verificar en cada entrega.